Sparse Quantum Simulation

CSC591/ECE592 Project Report

Blake Burgstahler[1] and Srikar Chundury[1]

[1]*North Carolina State University*

(Dated: November 28, 2022)

We aim to produce a numerical library which can analytically simulate the progression of quantum bits (qubits) through a quantum circuit by leveraging some key assumptions about the structure of gates once they are represented as Tensors (or Matrices). Existing libraries tend to lean on dense representations of these tensors, so this numerical library will leverage sparsity and be built up starting from custom data structures which exploit a few key assumptions about the structure to assist in speeding up computation while greatly reducing storage needs.

I. INTRODUCTION

Tensors can be thought of as multi-dimensional maps or functions. For example, a 2 dimensional tensor would be a matrix. A graph with tensors as vertices and "tensor methods" as edges is called a tensor network. Tensor methods could be contractions (matrix multiplication in 2D), decompositions/factorizations (PCA [8], SVD [3] in 2D) and more. Tensor networks have useful implications in applied mathematics, chemistry, physics, machine learning and many other fields.

Quantum circuits can be modelled as tensor networks not only to study and understand them better, but also to efficiently simulate them on classical computers. Fig. 1 shows one such way to adapt a circuit to a tensor network.

Efficient simulation of quantum circuits is important for two main reasons:

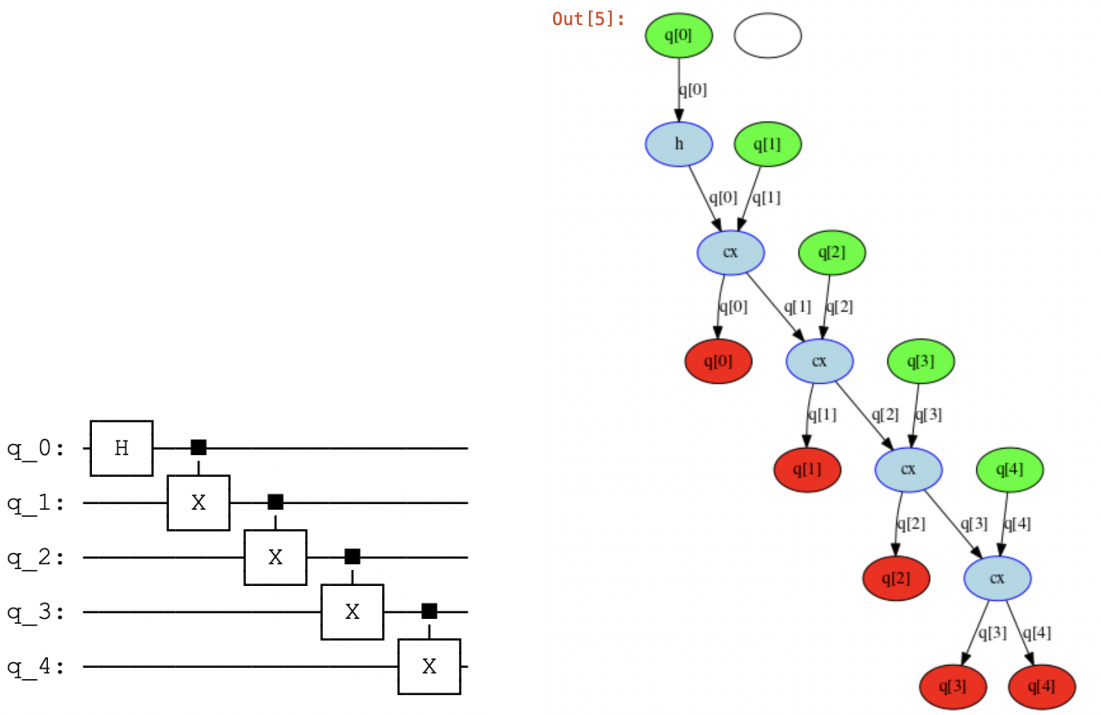
1. Quantum hardware is costly, and state preparation takes time which makes it infeasible to run multiple attempts while coming up new quantum algorithms,

2. Today's quantum computers suffer from problems like decoherence and noise which can add to inaccuracies in practice.

II. RELATED WORK

A. Tensor Network Contractions

A tensor is a generalization of vectors and matrices, and can easily be understood as a multidimensional array. A scalar being a zero dimensional tensor, a vector is a one dimensional tensor, and a matrix is a second order tensor. Hence, tensors are a mathematical



(a) Circuit Representation

(b) Tensor network representation

Figure 1: Two ways of representing the same circuit

concept that encapsulate and generalize the idea of multi-linear maps. A tensor contraction is an operation on a tensor in a finite-dimensional vector space. Therefore, it can be expressed as a sum of products of scalar components of different tensors. A tensor network is a countable collection of tensors connected by ”contractions”.

Classical simulation of quantum circuits is memory bound, namely, with every additional qubit the memory requirement doubles [4]. Hence, efficient contraction algorithms that ”evolve” through the tensor network become crucial for better performance.

Today's tensor network based quantum simulations involve a few steps:

1. Representation of a given quantum circuit as a tensor-network.
2. Local optimizations like decompositions to sub-networks.
3. Finding optimal contraction path [2].
4. Pair-wise contraction of tensors to arrive at a final state in case of a state simulator, or a unitary matrix in case of a unitary simulator.

B. Existing Tensor Libraries

Today's libraries that leverage tensor networks like quimb [6], qiskit [7] represent all tensors in dense formats, many of which can be passed directly to qiskit using the 'unitary' and 'UnitaryGate' functions included with qiskit. These dense data structures leave much to be desired as many of the gates (and even full circuits) used in quantum computing actually end up being dominated by 0 values with a sprinkling of ± 1 throughout - scaled by appropriate constants as necessary. Under this assumption, it doesn't always make sense for us to use dense tensor storage, so we began looking into existing approaches to handling sparse matrix storage. If we limit ourselves further, to only consider two dimensional tensors (in other words, matrices) there are a few very well know methods for sparse storage: Compressed Sparse Column (CSC), Compressed Sparse Row (CSR) and Coordinate Format (COO). However, it is always best practice to choose a data structure which will align with our needs best and none of these options would quite cut it.

Unitary simulation is nothing but chain matrix multiplications of all intermediate unitary matrices. Quimb does it more efficiently since it first computes the optimal path using "opt_einsum" and then multiplies unitary matrices in the discovered cost efficient order. But, Qiskit does not currrently do that, instead it evolves an identity matrix over a series of left-to-right unitary matrices in the order as they appear in a given quantum circuit. Further, Qiskit uses Numpy(dense) for this series of tensor "contractions".

III. DESIGN

As previously mentioned, the tensors in quantum tensor networks are quite sparse and follow specific patterns. Particularly, the pattern we chose to further exploit is that many of the nonzero values fall on some sort of diagonal. Therefore, our goal was to produce a numerical library which will leverage the sparsity introduced in the above-described simulation process and help achieve performance gains with respect to storage in memory and overall simulation time by purpose fitting our data structure to quantum computing use cases. In this way, we can both model sub-circuits as well as the overall circuit by considering their (hopefully) sparse representation as a unitary matrix – or tensor products of unitary matrices. However, since our intended qiskit interfaces expect dense matrix representations, had to pivot and elected simply build up a proof-of-concept library in C++ and leave the details of interfacing elsewhere for future work.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Figure 2: The Toffoli gate in matrix form. Notice, the upper left 6x6 submatrix is a diagonal and the lower right 2x2 is a single anti-diagonal.

With all of this in mind, we began discussions on how to draw inspiration from CSC, CSR, and COO in a way which leverages our assumption that in quantum the untitaries tend to be dominanted by values on diagonals. For example, the Toffoli gate can be represented by a $4x4$ matrix, and more specifically by only the places where we find values of 1. Namely, on two diagonals - one principal diagonal and one anti-diagonal as shown in Fig. 2 Many of the simplest gates we work with also follow this trend of values only falling on a select few diagonals.

Ideally, we'd be able to construct our sparse format in such a way that we can use the smallest combination of principal and anti-diagonals necessary to include all non-zero values, but the use of anti-diagonals was an additional degree of complexity that we elected to skip in favor of the simplicity of multiplication of principal diagonals. With this in mind, *every reference to 'diagonal' from here onward will be assumed to be in reference to the 'principal diagonals'.* By taking this simplification, we are still able to fully represent unitary matrices using marginally more total diagonals but have the added benefit of keeping matrix multiplication simple. Specifically, if every matrix we store is simply a collection of diagonals, the bulk of calculating a matrix product can be collapsed to appropriately multiplying pairs of diagonals. This collapses even further, as a matrix which is nonzero only on some diagonal is simply an operator which performs an element-wise shift to matrix elements followed by some element-wise multiplication to appropriately scale. An example of this operation over an entire matrix (a collection of diagonals) can be found in Fig. 3.

$$S = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}; \quad A = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 2 & 2 & 1 \\ 1 & 2 & 3 & 2 & 1 \\ 1 & 2 & 2 & 2 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Then,

$$SA = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 2 & 2 & 1 \\ 1 & 2 & 3 & 2 & 1 \\ 1 & 2 & 2 & 2 & 1 \end{pmatrix}; \quad AS = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 \\ 2 & 2 & 2 & 1 & 0 \\ 2 & 3 & 2 & 1 & 0 \\ 2 & 2 & 2 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix}.$$

Figure 3: Shift operations encoded in matrices. Notice That the order of the matrix multiplication corresponds to vertical or horizontal shift. Depending on the order.

It is worth recognizing that even dense matrices can still be represented using this approach. Analogous to the thinking that a matrix is simply a collection of column vectors, we will assume that the matrices we work with are collections of diagonals instead. However, we

choose this diagonal representation as it fits our assumption that many quantum operations can be encoded uses a minimal set of diagonals.

In summary, our design phase ended with these major assumptions:

1. Our Tensor Operators will all be Degree 2 (specifically, Matrices)
2. Generally speaking, these Matrices have more zero values than non-zero values.
3. Most often, the non-zero values fall on some diagonal pattern. For simplicity, we choose to assume that principal diagonals are easy to work with than anti-diagonals.
4. Working with only principal diagonals, any Matrix product can be written as a linear combination of the pairwise products of each of these diagonals.
5. The product of two diagonals is simply a shifting operation followed by element-wise multiplication.

IV. IMPLEMENTATION

First and foremost, the first half (or more) of this project was design oriented and the implementation of the above data structure is done entirely in C++ [1]

With the language chosen[2], we elected to attempt this implementation in two different ways. The first is only semi-sparse and assumes that each individual diagonal is essentially a dense vector - with appropriate sizing to reduce the size of off diagonals. In this way, we only need to store the diagonals which have non-zero values. We'll refer to this first approach as 'Sparse Diagonals'. The second approach takes this even further and adds the assumption that each of these vectors is itself sparse. In keeping with the spirit of CSR and CSC, we will refer to this second approach as 'Compressed Sparse Diagonals'.

A. Sparse Diagonals

Implemented as a C++11 class, 'Sparse Diagonal' encompasses two private members, 1) The principal diagonal index describing the position of the diagonal in any given square matrix, and 2) A vector of complex numbers that represents values in the diagonal. It is important to note that different diagonal indices represent diagonals of different lengths which are directly determined by the dimension of the matrix and the diagonal on which the element falls. Hence, this way of storing diagonals is more storage efficient than the DIA format [1] where the offsets are stored in a different array and the actual values are

[1] Unfortunately, while Blake has some experience in, he had not used it in years and Srikar had never used. So a solid chunk of actually implementing this code was either a refresher or (re)learning the language.

[2] Due to our limited C++ knowledge, we know there are many faux pas in how our code was written. Again, our goal was proof of concept, not provide production level code

stored in fixed length arrays. A method to compute the product of two diagonals (shift and element wise multiplication) has been implemented under "include/matrixKernels.cpp". This method is then called for all combinations of diagonals when two matrices are multiplied. This step involves multiple independent "new diagonal" computations (diagonal index A when multiplied with diagonal index B, results in a diagonal index of A+B).

B. Compressed Sparse Diagonals

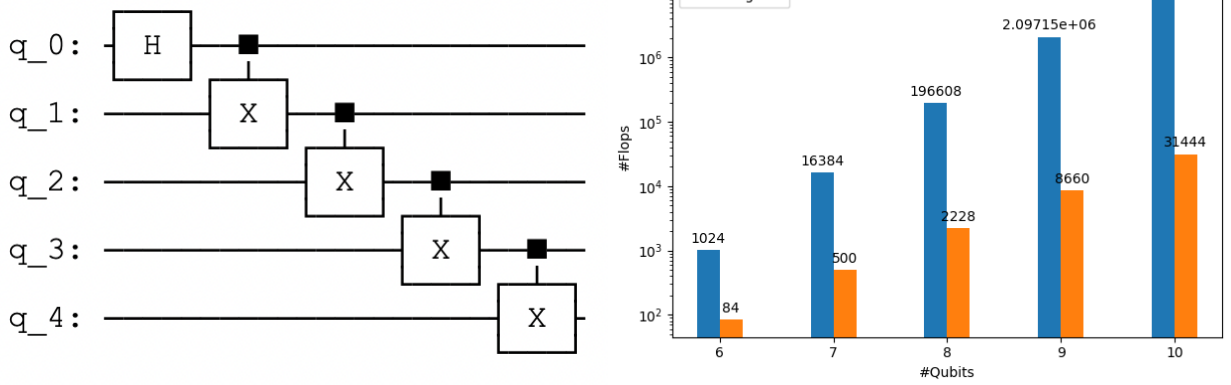
This implementation was built in parallel to the original sparse diagonals, and unfortunately ran into some major bugs in implementation that (as of writing this report) have not been fully handled. Given more time, they could likely be handled but due to time constraints must be left in a state of being a *Work in Progress*. With this in mind, all of the following analysis is specifically of the 'Sparse Diagonals' form.

V. TESTING FRAMEWORK

We test our approach on the supermarq benchmark suite's [9] ghz, hamiltonian, mermin bell circuits. Since, our primary concern is with efficient "contractions", we focus only on the unitary simulation of different sizes of these circuits. We also experiment with one ansatz borrowed from qiskit's QAOA textbook that finds the maxcut value for a square graph. "tests/tests.cpp" has various testcases covering all edge cases. As of now, we are not stable enough to expose our library as a python module to directly integrate with quimb or qiskit. Instead, we first export the intermediate matrices generated in qiskit as .npz files. We then import these files (using cnpy, a third party library) into our new data structure and then perform chain matrix multiplication using our kernels. "benchmarks/benchmarks.cpp" contains code for the same. We count the number of additions and multiplications performed within the chain of matrix multiplications using the $O(n^3)$ dense format and our new $O(n * d * d)$ Sparse Diagonal format where d is the number of diagonals.

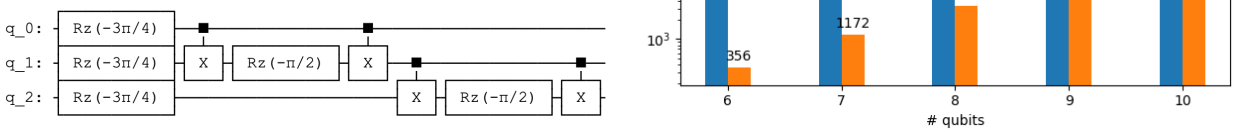
VI. RESULTS

As theoretically seen above, our approach (diagonal arithmetic) will compute the final unitary in significantly lesser number of additions and multiplications. These are confirmed in testing as described above. We can see these improvements in the following figures. Especially in case of Supermarq's Hamiltonian circuit Fig. 5a, which has a unitary representation of a perfect singular diagonal. We also see from Fig. 5b that dense kernel can be improved by around 4 orders of magnitude. Additional results can be found on in our git io page `https://pages.github.ncsu.edu/tnqs/sparse-tnqs/#4` but have been omitted here for the sake of brevity.



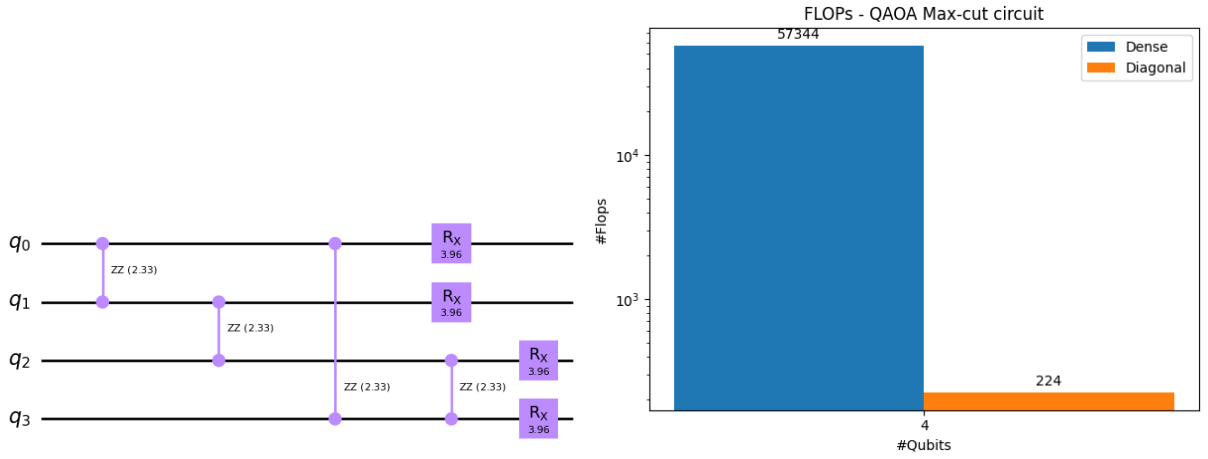
(a) GHZ circuit (4 qubit version) (b) Comparison (Dense/Numpy vs Sparse Diagonal)

Figure 4: Supermarq's GHZ circuit unitary simulation for different #qubits



(a) Hamiltonian circuit (3 qubit version) (b) Comparison (Dense/Numpy vs Sparse Diagonal)

Figure 5: Supermarq's Hamiltonian circuit unitary simulation for different #qubits



(a) Max cut circuit - without Hs (b) Comparison (Dense/Numpy vs Sparse Diagonal)

Figure 6: QAOA Max-cut for a square

VII. CONCLUSION

We observe that most unitary matrices involved in the process of quantum circuit simulation are "diagonal sparse" i.e. most non-zeros are present close to the main diagonal. We make use of it to create a custom data structure to store these matrices saving space, and write custom Matrix product kernels to reduce computational complexity from $O(n^3)$ to $O(d * d * n)$. We then perform multiple experiments (chain matrix multiplications) to simulate unitary evolution. As expected, we see that our approach beats the dense version by many orders. We hope that this work provides a basis to investigate further in this direction (diagonal/anti-diagonal). We can further save space using the compressed version as described above, which remains a work in progress. Our approaches can then be compared against DIA format [1] for benchmarks from Suite Sparse [5]. Evidence seems to suggest that a compressed diagonal version will not only be more space efficient but also continue to improve computational complexity given typical quantum unitary matrices. Ideally, as most of today's quantum simulation frameworks expose python libraries, we plan to expose our library as a python package for use with other libraries (for example, opt_einsum [2]).

VIII. ACKNOWLEDGEMENTS

We thank Dr. Frank Mueller for his guidance and support throughout the semester.

---

[1] Dia format. `https://matteding.github.io/2019/04/25/sparse-matrices/`. Accessed: 2022-11-21.

[2] Daniel G. a. Smith and Johnnie Gray. opt_einsum - a python package for optimizing contraction order for einsum-like expressions. *Journal of Open Source Software*, 3(26):753, 2018.

[3] Steve Brunton. Singular value decomposition (svd): Overview, 2020.

[4] Mircosoft Corporation. Operations on multiple qubits, 2022.

[5] Timothy A. Davis and Yifan Hu. The university of florida sparse matrix collection. *ACM Trans. Math. Softw.*, 38(1), dec 2011.

[6] Johnnie Gray. quimb: a python library for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018.

[7] IBM. Learn quantum computation using qiskit, 2021.

[8] J. Nathan Kutz. Principal component analysis (pca), 2016.

[9] Teague Tomesh, Pranav Gokhale, Victory Omole, Gokul Subramanian Ravi, Kaitlin N. Smith, Joshua Viszlai, Xin-Chuan Wu, Nikos Hardavellas, Margaret R. Martonosi, and Frederic T. Chong. Supermarq: A scalable quantum benchmark suite, 2022.